XARM使用手册-天轶（对外）

1. 部署

安装XARM环境：XARM_env_release.tar.gz

```
代码块
1   tar -zxvf XARM_env_release.tar.gz
2   cd XARM_env_release
3   bash install.sh
```

安装XARM本体框架： XARM_release.tar.gz

```
代码块
1   tar -zxvf XARM_release.tar.gz
2   cd XARM_release
3   bash install.sh
```

2. 整体介绍

XARM基于ros2control框架进行开发，相关概念详细请参考ros2control官方文档

Welcome to the ros2_control documentation - Humble!

框架按照ros2control的划分主要分为两层，上层Controller层负责各种运动生成类的功能，下层Hardware层负责控制相关的功能。上下两层可组合使用，例如上层选择关节空间指令控制器，直接给定机械臂希望达到的关节角度，下层如果选择高刚度控制器，则机器人会以尽可能的方式精准达到给定位置；如果下层选择使用柔顺控制器，则机器人会保持一定的对外界柔顺能力。

本框架在机器人上的部署位置为

/home/ubuntu/XARM

使用前需要运行

```
代码块
1   source /home/ubuntu/XARM/install/setup.bash
```

日志文件位于

/home/ubuntu/XARM_logs

3. 控制器说明

3.1 jointspace_arm_L_controller

特点：接收关节空间指令，内部进行实时速度、加速度、jerk平滑以及限位检查后进行执行

左手关节空间控制器,

接收话题

```
代码块
1   /jointspace_commands_L
```

类型

```
代码块
1   std_msgs::msg::Float64MultiArray
```

话题为需要为一个长度为7的数组，分别给定左手7个关节的期望达到的角度

示例

```
代码块
1   ros2 topic pub /jointspace_commands_L std_msgs/msg/Float64MultiArray 'data:
    [0.0, 1.18, 0.0, -1.3, 0.0, -0.13, 0.18]'
```

接收action

接口名：

```
代码块
1   /jointspace_arm_L_controller/jointspace
```

接口的类型:

```
代码块
1   eai_manipulator_msgs/action/JointSpace
```

其定义为

```
代码块
1   float64[] target_positions
2   ---
3   bool success
4   string result_msg
5   ---
6   float64[] current_positions
7   string feedback_msg
```

- target_positions : 目标关节角，长度需要为7，且每个目标值都在限位范围之内
- success : 运行是否成功
- result_msg : 运行完成后的返回信息
- current_positions : 运行过程中的规划器规划值
- feedback_msg : 中间反馈信息

示例：

```
代码块
1   ros2 action send_goal /jointspace_arm_L_controller/jointspace
    eai_manipulator_msgs/action/JointSpace "{target_positions: [0.0, 1.18, 0.0,
    -1.3, 0.0, -0.13, 0.18]}"
```

注意事项：

action接口和topic接口只能同时使用其中之一，当使用action时需要停止发送topic

主要参数：

- vel_limits： 运动的速度， 可在线更改
- acc_limits： 运动的加速度，可在线更改
- jerk_limits： 运动的加加速度，可在线更改

未出现在本文档中的参数不建议更改

举例

代码块

```
#将各个关节的运动速度分别设置为对应值
ros2 param set /jointspace_arm_L_controller vel_limits '[1.1, 1.2, 1.3, 1.4, 1.5, 1.6, 1.7]'

#查看更改结果
ros2 param get /jointspace_arm_L_controller vel_limits

```

建议acc_limits、jerk_limits保留配置文件中的默认值

注意事项

- 该控制器不包含防碰撞能力
- 设置vel_limits不应超过物理机构执行能力
- 下发指令应处于各个关节限位之内

运行状态以及报错

信息源：source = JointspaceRuckigController_left_arm

信息代码	类型	解释
100101	info	内部规划器状态发生变化时发生，一般用来指示收到新的目标数据
100102	info	内部规划器运行完成时发生，可用于指令的到位判断

600100	error	内部规划器运行报错，通常为优化问题不可行
600101	error	关节指令超上限位
600102	error	关节指令超下限位

3.2 jointspace_arm_R_controller

特点：同3.1 但是控制右手

右手关节空间控制器, 话题名为

```
代码块
1   /jointspace_commands_R
```

示例

```
代码块
1   ros2 topic pub /jointspace_commands_R std_msgs/msg/Float64MultiArray 'data:
    [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]'
```

信息源：source = JointspaceRuckigController_right_arm

其余同 3.1

接口名：

```
代码块
1   /jointspace_arm_R_controller/jointspace
```

范例：

```
代码块
```

```
ros2 action send_goal /jointspace_arm_R_controller/jointspace
eai_manipulator_msgs/action/JointSpace "{target_positions: [0.0, -1.18, 0.0,
-1.3, 0.0, -0.13, 0.18]}"
```

3.3 endpose_wholebody_controller

特点：手臂运动时会使用腰腿关节以增大工作空间范围

代码块

```
指令话题名称：/endposetarget_Dual
消息msg格式：
# 左臂目标位姿信息
geometry_msgs/Pose target_l        # 目标位姿 T_A_B（从A系到B系的变换）
string from_frame_l                # A系名称（参考系）
string to_frame_l                  # B系名称（目标系）
float32 offset_x_l                 # 末端参考点X方向偏移量
float32 offset_y_l                 # 末端参考点Y方向偏移量
float32 offset_z_l                 # 末端参考点Z方向偏移量

# 右臂目标位姿信息
geometry_msgs/Pose target_r        # 目标位姿 T_A_B（从A系到B系的变换）
string from_frame_r                # A系名称（参考系）
string to_frame_r                  # B系名称（目标系）
float32 offset_x_r                 # 末端参考点X方向偏移量
float32 offset_y_r                 # 末端参考点Y方向偏移量
float32 offset_z_r                 # 末端参考点Z方向偏移量

bool is_dual_arm_syn               # 是否启用双臂同步运动（true=同步，false=独立）

可配置参数列表如下：
vel_limits: [0.3,0.3,0.3,0.3,1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0,
1.0, 1.0, 1.0, 1.0]
acc_limits: [1.0,1.0,1.0,1.0,10.0, 10.0, 10.0, 10.0, 10.0, 10.0, 10.0, 10.0,
10.0, 10.0, 10.0, 10.0, 10.0, 10.0]
jerk_limits: [10.0,10.0,10.0,10.0,50.0, 50.0, 50.0, 50.0, 50.0, 50.0, 50.0,
50.0, 50.0, 50.0, 50.0, 50.0, 50.0, 50.0]
joint_home_pos: [0.0,0.0,0.0,0.0,0.0, 1.18, 0.0, -1.3, 0.0, -0.13, 0.18, 0.0,
-1.18, 0.0, -1.3, 0.0, -0.13, 0.18]
collision_r:
[0.225,0.225,0.0975,0.175,0.045,0.045,0.045,0.045,0.045,0.061,0.081,0.045,0.045
,0.045,0.045,0.045,0.061,0.081,0.065] # 碰撞小球半径，身体2，关节7+7，freeball1
collision_freeBall_pos: [0.0, 0.0, 0.0,0.10] # 自由碰撞小球xyz半径(半径做抽象参数)
OriWeight: [1.0, 1.0, 1.0] #优化中姿态控制权重 0.0～1.0
waistWeight: [1.0, 1.0, 1.0, 1.0] # 优化中腰部控制权重 0.0～1.0
waistPos: [0.04, 0.78, 0.0, 0.0] # 优化中腰部位姿 xz pitch yaw
```

```
dis_err_bound: 0.2
ori_err_bound: 1.0
otg_r_step: 0.0055
otg_p_step: 0.0055
redundant_degrees_left: 0.0 # 左臂冗余信息
redundant_degrees_right: 0.0 #右臂冗余信息
redundant_degrees_weight: 20.0 # 优化任务权重
关节顺序为:
                "first_leg_pitch_joint",
                "second_leg_pitch_joint",
                "waist_pitch_joint",
                "body_yaw_joint",
                "shoulder_pitch_l_joint",
                "shoulder_roll_l_joint",
                "shoulder_yaw_l_joint",
                "elbow_pitch_l_joint",
                "elbow_yaw_l_joint",
                "wrist_pitch_l_joint",
                "wrist_roll_l_joint",
                "shoulder_pitch_r_joint",
                "shoulder_roll_r_joint",
                "shoulder_yaw_r_joint",
                "elbow_pitch_r_joint",
                "elbow_yaw_r_joint",
                "wrist_pitch_r_joint",
                "wrist_roll_r_joint"

```

3.4 jointspace_body_controller

特点：腰腿四个关节空间指令，内部进行实时速度、加速度、jerk平滑以及限位检查后进行执

腰腿关节空间控制器, 话题名为

代码块

```
/jointspace_commands_body
```

数据长度为4，

示例

代码块

```
ros2 topic pub /jointspace_commands_body std_msgs/msg/Float64MultiArray 'data:
[0.0, 0.0, 0.0, 0.0]'
```

其余同3.1

3.5 jointspace_body3joints_controller

特点：腰腿3个关节空间指令，内部进行实时速度、加速度、jerk平滑以及限位检查后进行执

本控制器与jointspace_body_controller的差别是只控制器前3个关节，也就是"first_leg_pitch_joint", "second_leg_pitch_joint", "waist_pitch_joint"

腰腿关节空间控制器, 话题名为

代码块

```
/jointspace_commands_body
```

数据长度为3，

示例

代码块

```
ros2 topic pub /jointspace_commands_body std_msgs/msg/Float64MultiArray 'data:
[0.0, 0.0, 0.0]'
```

其余同3.1

3.6 jointspace_head_controller

特点：头部3个关节空间指令，内部进行实时速度、加速度、jerk平滑以及限位检查后进行执

头部关节空间控制器，话题名为

代码块

```
/jointspace_commands_head
```

数据长度为3，

示例

```
代码块
1   ros2 topic pub /jointspace_commands_head std_msgs/msg/Float64MultiArray 'data:
    [0.0, 0.0, 0.0]'
```

其余同3.1

3.7 jointspace_waist_pitch_controller

特点：腰部俯仰 关节空间指令，内部进行实时速度、加速度、jerk平滑以及限位检查后进行执

腰部pitch关节空间控制器, 话题名为

```
代码块
1   /jointspace_commands_waist_pitch
```

数据长度为1，

示例

```
代码块
1   ros2 topic pub /jointspace_commands_waist_pitch std_msgs/msg/Float64MultiArray
    'data: [0.0]'
```

其余同3.1

3.8 jointspace_waist_yaw_controller

特点：腰部偏航 关节空间指令，内部进行实时速度、加速度、jerk平滑以及限位检查后进行执

腰部yaw关节空间控制器, 话题名为

```
代码块
1   /jointspace_commands_waist_yaw
```

数据长度为1，

其余同3.1

示例

```
ros2 topic pub /jointspace_commands_waist_yaw std_msgs/msg/Float64MultiArray
'data: [0.0]'
```

3.9 jointspace_ruckig_qp_controller

特点：接收双臂关节空间指令，内部进行实时速度、加速度、jerk平滑以及限位检查后进行执，同时带自身防碰撞，双臂之间防碰撞

双臂关节空间控制器，带与身体的防碰撞，使用方法同3.1中topic部分，但不支持aciton接口

在确定不会发生于自身碰撞的情况下，建议使用jointspace_arm_L_controller 和 jointspace_arm_R_controller 而非本控制器

接收话题

代码块

```
/jointspace_commands_Dual
```

类型

代码块

```
std_msgs::msg::Float64MultiArray
```

长度为14个数组，前七个位左手，后7个为右手

3.10 endpose_body_controller

特点：腰腿笛卡尔空间控制器，可直接给定腰部waist_yaw_link的目标位姿，顺序为[x z pitch yaw]

天轶身体控制器，用以控制天轶的腿和腰部

接收话题

代码块

```
/endposetarget_Body
```

话题类型

代码块

```
std_msgs::msg::Float64MultiArray
```

该控制器接收一个长度为4的数组，四个数分别为[x, z, pitch, yaw],表示腰部相对于基座的x，z坐标以及pitch角、yaw角。

示例

代码块

```
ros2 topic pub /endposetarget_Body std_msgs/msg/Float64MultiArray 'data: [0.0, 0.6, 0.0, 0.0]'
```

接收action

代码块

```
/endpose_body_controller/jointspace
```

接口的类型:

代码块

```
eai_manipulator_msgs/action/JointSpace
```

其定义为

代码块

```
float64[] target_positions
---
bool success
string result_msg
---
float64[] current_positions
```

```
7   string feedback_msg
```

其中各项为:

- target_positions : 目标关节角，长度需要为4，且每个目标值都在限位范围之内
- success : 运行是否成功
- result_msg : 运行完成后的返回信息
- current_positions : 运行过程中的规划器规划值
- feedback_msg : 中间反馈信息

参数

主要参数:

- vel_limits： 运动的速度， 可在线更改
- acc_limits： 运动的加速度，可在线更改
- jerk_limits： 运动的加加速度，可在线更改

未出现在本文档中的参数不建议更改

注意事项

- 本控制器不具备多轴同步功能，因此不能实现腰部的直接笛卡尔运动
- IK求解失败、目标位置超出可能范围，该控制器会保持原位

3.11 endpose_head_controller

头部末端控制器，用于控制头部关节的末端姿态

接收话题

代码块

```
1   /endposetarget_Head
```

话题类型

代码块

```
std_msgs::msg::Float64MultiArray
```

接收一个长度为3的数组，分别表示从腰部到头部所期望的ZYX欧拉角，也就是 **Roll、 Yaw、 Pitch**

示例

代码块

```
ros2 topic pub //endposetarget_Head std_msgs/msg/Float64MultiArray 'data:
[0.0, 0.0, 0.0]'
```

3.12 endpose_single_arm_qp_L_controller

特点：左手末端笛卡尔空间控制器，局部响应式目标跟踪，若设置目标偏离当前太多，会拒绝执行并有相应报警信息，内带防碰撞功能

左手末端笛卡尔空间局部控制器，给定末端期望位姿，该控制器会尽可能达到期望位姿

接收话题

代码块

```
/endposetarget_L
```

话题类型eai_manipulator_msgs/msg/ArmTargetPose

代码块

```
std_msgs/Header header

geometry_msgs/Pose target       # 目标位姿 T_A_B 从A系的B系
string from_frame               # A系名称
string to_frame                 # B系名称
float32 offset_x                # 末端参考偏移量
float32 offset_y
float32 offset_z


```

from_frame 和 to_frame 共同决定了target的含义，例如

若 from_frame = 'base_footprint'

to_frame = 'left_tcp_link'

则target的含义为从基座到末端left_tcp_link的期望位姿。

若 from_frame = 'waist_yaw_link'

to_frame = 'left_tcp_link'

则target的含义为从腰部到末端left_tcp_link的期望位姿。

offset 表示在末端坐标系下的参考点偏移

接收Action

自V0.5版本开始支持action接口

接口名：

```
代码块
1   /endpose_single_arm_qp_L_controller/endPosSingleTarget
```

接口的类型:

```
代码块
1   eai_manipulator_msgs/action/EndPosSingleTarget
```

其定义为

```
代码块
1   eai_manipulator_msgs/ArmTargetPose target
2   ---
3   bool success
4   string result_msg
5   ---
6   float64[] current_err
7   string feedback_msg
8
```

- eai_manipulator_msgs/ArmTargetPose target ：继承于topic使用的自定义话题 见3.5
- success : 运行是否成功

- result_msg : 运行完成后的返回信息
- current_err : 运行过程中6D位姿误差 xyz rxryrz；
- feedback_msg : 中间反馈信息

示例：

代码块

```
ros2 action send_goal /endpose_single_arm_qp_L_controller/endPosSingleTarget
eai_manipulator_msgs/action/EndPosSingleTarget \
'{
  "target": {
    "header": {"frame_id": "waist_pitch_link"},
    "target": {
      "position": {"x": 0.0, "y": -0.2, "z": -0.20},
      "orientation": {"x": 0.0, "y": 0.0, "z": 0.0, "w": 1.0}
    },
    "from_frame": "waist_pitch_link",
    "to_frame": "right_tcp_link",
    "offset_x": 0.0, "offset_y": 0.0, "offset_z": 0.0
  }
}'
```

注意事项：

action接口和topic接口只能同时使用其中之一，当使用action时需要停止发送topic

参数

若非特意声明，参数均不支持在线更改，需修改配置文件后重新启动。

未出现在本文档中的参数不建议自行更改。

- vel_limits 关节运动速度
- acc_limits 关节运动加速度
- dis_err_bound 末端跟踪距离误差限，期望位姿需靠近当前末端到此距离之内
- ori_err_bound 末端跟踪角度误差线，期望位姿需靠近当前末端到此角度之内
- collision_r 自碰撞小球半径 [10维] 身体2，胳膊7，freeball 1
- collision_freeBall_pos: 自由小球坐标与半径 [0.0, 0.0, 0.0,0.10] （希望添加碰撞小球用这个就好），尽可能不要将小球xyz 设置在机械臂物理位置内
- OriWeight: [1.0, 1.0, 1.0] 优化中姿态控制权重 0.0～1.0 ，如希望做 5dof(直线位姿) 4dof(平面位姿)控制时可将对应权重设置为 0.0 ，可以配合工具坐标系使用

- redundant_degrees 优化中冗余关节[shoulder_yaw_oint]参考角度，该关节角度可改变零空间臂角
- redundant_degrees_weight 优化中冗余关节[shoulder_yaw_oint]参考角度所占到的权重 推荐0.0 ～500.0 该参数越大则跟踪参考臂角的效果越好，过大会牺牲部分末端精度
- dis_threshold: Action 中 停止的距离阈值
- ori_threshold: Action 中 停止的角度阈值
- step_threshold: Action 中 停止的优化次数阈值

注意事项

- from_frame建议值为base_footprint 或 waist_yaw_link
- to_frame需要与末端固连，建议值为left_tcp_link
- 期望位姿不满足误差限时会保持不动，注意处理报错和警告信息

运行状态及报错

信息代码	类型	解释
300100	warn	给定目标位姿超出跟踪限
30025x	warn	关节达到最小限位
30026x	warn	关节达到最大限位
600120	error	from_frame设置错误
600130	error	to_frame设置错误

3.13 endpose_single_arm_qp_R_controller

特点：同3.11 给右手用

右手末端笛卡尔空间局部控制器，

话题名为

```
代码块
1   /endposetarget_R
```

其余同3.5

Action 名称为：

```
代码块
1   /endpose_single_arm_qp_R_controller/endPosSingleTarget
```

3.14 endpose_single_arm_qpik_L_controller

特点：左手末端笛卡尔空间控制器，局部响应式目标跟踪，若设置目标偏离当前太多，会拒绝执行并有相应报警信息，内带防碰撞功能

左手末端笛卡尔空间局部控制器，给定末端期望位姿，该控制器会尽可能达到期望位姿

接收话题

```
代码块
1   /endposetarget_L
```

话题类型eai_manipulator_msgs/msg/ArmTargetPose

```
代码块
1   std_msgs/Header header
2
3   geometry_msgs/Pose target       # 目标位姿 T_A_B 从A系的B系
4   string from_frame               # A系名称
5   string to_frame                 # B系名称
6   float32 offset_x                # 末端参考偏移量
7   float32 offset_y
8   float32 offset_z
9
10
```

该控制器为了与其他控制器保持接口一致，复用了eai_manipulator_msgs/msg/ArmTargetPose但是其中只有target这个数据有效

该控制器会额外接收话题以完成冗余控制，会根据姿态任务和关节控制任务的权重进行相应组合

```
代码块
1   /jointspace_commands_L
```

参数

- vel_limits： 运动的速度， 可在线更改
- from_frame : 目标值的源坐标系， 默认值为"base"
- to_frame： 目标值的target坐标系, 默认值为"left_tcp_link"
- EE_offset : 末端参考偏移，数据格式为 [x,y,z,r,p,y]， 默认值全0
- Cspace_weight： 关节空间参考位置权重 长度为7 默认值为 [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
- position_weight : 位置任务权重
- orientation_weight : 姿态任务权重

该控制器需要通过更改frome_frame、to_frame、EE_offset这几个参数来灵活设置参考目标

额外功能

1. 该控制器会发送一个名为 “EE_with_offset_L”的tf来指示当前的参考末端位置

3.15 endpose_dual_arm_qp_controller

双臂末端笛卡尔空间局部控制器，用于实现双臂同步的笛卡尔末端运动

接收话题

```
代码块
```

```
/endposetarget_Dual
```

话题类型eai_manipulator_msgs/msg/DualArmTargetPose

代码块

```
std_msgs/Header header

# 左臂目标位姿信息
geometry_msgs/Pose target_l        # 目标位姿 T_A_B（从A系到B系的变换）
string from_frame_l                # A系名称（参考系）
string to_frame_l                  # B系名称（目标系）
float32 offset_x_l                 # 末端参考点X方向偏移量
float32 offset_y_l                 # 末端参考点Y方向偏移量
float32 offset_z_l                 # 末端参考点Z方向偏移量

# 右臂目标位姿信息
geometry_msgs/Pose target_r        # 目标位姿 T_A_B（从A系到B系的变换）
string from_frame_r                # A系名称（参考系）
string to_frame_r                  # B系名称（目标系）
float32 offset_x_r                 # 末端参考点X方向偏移量
float32 offset_y_r                 # 末端参考点Y方向偏移量
float32 offset_z_r                 # 末端参考点Z方向偏移量

bool is_dual_arm_syn               # 是否启用双臂同步运动（true=同步，false=独立）

```

这里的坐标系数据含义参见3.11，需共同提供左手和右手的期望位姿。

当开启双臂同步 （is_dual_arm_syn = true）时，会以左手的位姿数据作为参考运动右手。

参数

若非特意声明，参数均不支持在线更改，需修改配置文件后重新启动。

未出现在本文档中的参数不建议自行更改。

- TODO

注意事项

- from_frame建议值为base_footprint 或 waist_yaw_link
- to_frame需要与末端固连，建议值为left_tcp_link
- TODO

3.16 moveit_left_arm_controller、 moveit_right_arm_controller

用于moveit使用，当希望使用moveit相关功能时需要激活该控制器

4. Moveit使用说明

4.1 关节空间控制

4.1.1 接口类型

ROS Service

4.1.2 关节数据下发服务名称

代码块

```
/joint_space_control
```

4.1.3 数据格式

代码块

```
ros2: cm_msgs::srv::CommonStreamRos2
```

4.1.3.1 请求数据格式

代码块

```
request:
    msg_id: ''
    json_data:{
    "group": "dual_arm",  //dual_arm（双臂）  left_arm/right_arm(单臂)
    "vel_scale": 0.1,
    "acc_scale": 0.1,
    "waypoints": [
    {
    "shoulder_pitch_l_joint": 0, //左臂关节角度（弧度）
    "shoulder_roll_l_joint": 0,
    "shoulder_yaw_l_joint": 0,
    "elbow_pitch_l_joint": 0,
```

```
      "elbow_yaw_l_joint": 0,
      "wrist_pitch_l_joint": 0,
      "wrist_roll_l_joint": 0,
      "shoulder_pitch_r_joint": 0, //右臂关节角度（弧度）
      "shoulder_roll_r_joint": 0,
      "shoulder_yaw_r_joint": 0,
      "elbow_pitch_r_joint": 0,
      "elbow_yaw_r_joint": 0,
      "wrist_pitch_r_joint": 0,
      "wrist_roll_r_joint": 0,
      }
      ]
      }
      data: []
```

4.1.3.2 响应数据格式

代码块

```
response:
    result: true //指令接收成功
```

4.1.3.3 使用案例

代码块

```
source ~/XARM/install/setup.bash
ros2 launch tianyi2_bringup tianyi2.launch.py hardware:=real
 (gui:=True 会启动rviz， 默认值为false
hardware:=real 会启动真机模式，默认值为sim
如果是天工 ros2 launch tiangong2pro_bringup tiangong2.launch.py)

ros2 control auto_switch_mode --enable (自动切换开启后，无需手动switch_controller)
```

- 双臂多点轨迹模式（入参只传单点时可作为单目标点关节规划使用）

单点：

代码块

```
ros2 service call /joint_space_control cm_msgs/srv/CommonStreamRos2 "msg_id: ''
json_data: '{
  \"group\": \"dual_arm\",
  \"vel_scale\": 0.5,
  \"acc_scale\": 0.5,
```

```
  \"waypoints\": [
    {
      \"shoulder_pitch_l_joint\": -0.30000, \"shoulder_roll_l_joint\": 0.30000, \"shoulder_yaw_l_joint\": 0.20000,
      \"elbow_pitch_l_joint\": -1.0000, \"elbow_yaw_l_joint\": -0.20000, \"wrist_pitch_l_joint\": -0.20000, \"wrist_roll_l_joint\": -0.10000,
      \"shoulder_pitch_r_joint\": -0.30000, \"shoulder_roll_r_joint\": -0.30000, \"shoulder_yaw_r_joint\": -0.20000,
      \"elbow_pitch_r_joint\": -1.0000, \"elbow_yaw_r_joint\": 0.20000, \"wrist_pitch_r_joint\": -0.20000, \"wrist_roll_r_joint\": -0.10000
    }
  ]
}'
data: []"
```

多点

代码块

```
ros2 service call /joint_space_control cm_msgs/srv/CommonStreamRos2 "msg_id: ''
json_data: '{
  \"group\": \"dual_arm\",
  \"vel_scale\": 0.5,
  \"acc_scale\": 0.5,
  \"waypoints\": [
    {
      \"shoulder_pitch_l_joint\": -0.30000, \"shoulder_roll_l_joint\": 0.30000, \"shoulder_yaw_l_joint\": 0.20000,
      \"elbow_pitch_l_joint\": -0.40000, \"elbow_yaw_l_joint\": -0.20000, \"wrist_pitch_l_joint\": -0.20000, \"wrist_roll_l_joint\": -0.10000,
      \"shoulder_pitch_r_joint\": -0.30000, \"shoulder_roll_r_joint\": -0.30000, \"shoulder_yaw_r_joint\": -0.20000,
      \"elbow_pitch_r_joint\": -0.40000, \"elbow_yaw_r_joint\": 0.20000, \"wrist_pitch_r_joint\": -0.20000, \"wrist_roll_r_joint\": -0.10000
    },
    {
      \"shoulder_pitch_l_joint\": -0.50000, \"shoulder_roll_l_joint\": 0.50000, \"shoulder_yaw_l_joint\": 0.40000,
      \"elbow_pitch_l_joint\": -0.60000, \"elbow_yaw_l_joint\": -0.40000, \"wrist_pitch_l_joint\": -0.40000, \"wrist_roll_l_joint\": -0.20000,
      \"shoulder_pitch_r_joint\": -0.50000, \"shoulder_roll_r_joint\": -0.50000, \"shoulder_yaw_r_joint\": -0.40000,
      \"elbow_pitch_r_joint\": -0.60000, \"elbow_yaw_r_joint\": 0.40000, \"wrist_pitch_r_joint\": -0.40000, \"wrist_roll_r_joint\": -0.20000
    },
    {
```

```
      \"shoulder_pitch_l_joint\": -0.20000, \"shoulder_roll_l_joint\": 0.20000, \"shoulder_yaw_l_joint\": 0.10000,
      \"elbow_pitch_l_joint\": -0.30000, \"elbow_yaw_l_joint\": -0.10000, \"wrist_pitch_l_joint\": -0.10000, \"wrist_roll_l_joint\": 0.00000,
      \"shoulder_pitch_r_joint\": -0.20000, \"shoulder_roll_r_joint\": -0.20000, \"shoulder_yaw_r_joint\": -0.10000,
      \"elbow_pitch_r_joint\": -0.30000, \"elbow_yaw_r_joint\": 0.10000, \"wrist_pitch_r_joint\": -0.10000, \"wrist_roll_r_joint\": 0.00000
    }
  ]
}'
data: []"
```

- 左臂多点轨迹模式（入参只传单点时可作为单目标点关节规划使用）

代码块

```
ros2 service call /joint_space_control cm_msgs/srv/CommonStreamRos2 "msg_id: ''
json_data: '{
  \"group\": \"left_arm\",
  \"vel_scale\": 0.5,
  \"acc_scale\": 0.5,
  \"waypoints\": [
    {
      \"shoulder_pitch_l_joint\": -0.30000, \"shoulder_roll_l_joint\": 0.30000, \"shoulder_yaw_l_joint\": 0.20000,
      \"elbow_pitch_l_joint\": -1.0000, \"elbow_yaw_l_joint\": -0.20000, \"wrist_pitch_l_joint\": -0.20000, \"wrist_roll_l_joint\": -0.10000
    }
  ]
}'
data: []"
```

- 右臂多点轨迹模式（入参只传单点时可作为单目标点关节规划使用）

代码块

```
ros2 service call /joint_space_control cm_msgs/srv/CommonStreamRos2 "msg_id: ''
json_data: '{
  \"group\": \"right_arm\",
  \"vel_scale\": 0.5,
  \"acc_scale\": 0.5,
  \"waypoints\": [
    {
      \"shoulder_pitch_r_joint\": -0.30000, \"shoulder_roll_r_joint\": -0.30000, \"shoulder_yaw_r_joint\": -0.20000,
```

```
      \"elbow_pitch_r_joint\": -1.0000, \"elbow_yaw_r_joint\": 0.20000,
  \"wrist_pitch_r_joint\": -0.20000, \"wrist_roll_r_joint\": -0.10000
    }
  ]
}'
data: []"
```

4.1.4 执行结果上报服务名称

代码块

```
/joint_action_report
```

4.1.5 数据格式

代码块

```
ros2: cm_msgs::srv::CommonStreamRos2
```

4.1.5.1 请求数据格式

moveit异常上报error code : Moveit Error Code

代码块

```
request:
    msg_id: ''
    json_data:{
        "action_name": "motion_result",
        "result": 1    //成功=1 失败=0
        "error_code": 1  //详见error code 说明
    }
    data: []
```

4.1.5.2 响应数据格式

代码块

```
response:
    result: true //指令接收成功
```

4.2 笛卡尔空间控制

4.2.1 接口说明

当前接口基于 ROS Service 实现面向机械臂笛卡尔空间下的运动控制，包括单臂、双臂配置下单个和多个目标点序列，同时该接口支持目标点在机器人base坐标系和末端坐标系下...

通过两个服务接口分别实现接收waypoints和执行结果上报。

4.2.2 接口类型

ROS 2 Service

4.2.3 waypoints下发服务名称

代码块

```
/dual_arm_waypoints
```

4.2.4 数据格式

代码块

```
ros2: cm_msgs::srv::CommonStreamRos2
```

4.2.4.1 请求数据格式

代码块

```
request:
    msg_id: ''
    json_data:{
    "group": "dual_arm", //dual_arm left_arm right_arm
    "frame": "waist_yaw_link",  //waist_yaw_link  (tianyi,tiangong2pro) waist_pitch_link(dex,evt2)
    "mode": "plan_and_execute" // 单臂异步控制  plan / execute / plan_and_execute
    "use_cartesian_path": false, //是否使用纯直线笛卡尔路径规划 仅支持单臂
    "left_motion_id": 0,
    "right_motion_id": 0,
    "vel_scale": 0.1,
    "acc_scale": 0.1,
    "left_waypoints":[[x, y, z, x, y, z, w],[x, y, z, x, y, z, w]],
    "right_waypoints":[[x, y, z, x, y, z, w],[x, y, z, x, y, z, w]]
    }
    data: []
```

4.2.4.2 响应数据格式

代码块

```
response:
    result: true //指令接收成功
```

4.2.4.3 使用案例

代码块

```
source ~/XARM/install/setup.bash
ros2 launch tianyi2_bringup tianyi2.launch.py gui:=True hardware:=real

 (gui:=True 会启动rviz， 默认值为false
hardware:=real 会启动真机模式，默认值为sim
如果是天工 ros2 launch tiangong2pro_bringup tiangong2.launch.py)

ros2 control switch_controllers --activate moveit_left_arm_controller
moveit_right_arm_controller
```

需注意的是frame： waist_yaw_link 对应机器为（tianyi,tiangong2pro）系列； waist_pitch_link对应机器为(dex,evt2) 系列

代码块

```
ros2 service call /dual_arm_waypoints cm_msgs/srv/CommonStreamRos2 "msg_id: ''
json_data: '{\"group\": \"dual_arm\",\"frame\":
\"waist_pitch_link\",\"left_motion_id\": 0, \"right_motion_id\": 0,
\"left_waypoints\": [[0.26954124421585335, 0.3736418954013253,
0.10457946313346808, -0.1529562230220876, -0.6448902522919694,
-0.1707295374594559, 0.7290901051149286],[0.28954124421585335,
0.3736418954013253, 0.10457946313346808, -0.1529562230220876,
-0.6448902522919694, -0.1707295374594559, 0.7290901051149286]],
\"right_waypoints\": [[0.26954124421585335, -0.3736418954013253,
0.10457946313346808, 0.1529562230220876, -0.6448902522919694,
0.1707295374594559, 0.7290901051149286],[0.28954124421585335,
-0.3736418954013253, 0.10457946313346808, 0.1529562230220876,
-0.6448902522919694, 0.1707295374594559, 0.7290901051149286]]}'
data: []"


ros2 service call /dual_arm_waypoints cm_msgs/srv/CommonStreamRos2 "msg_id: ''
json_data: '{\"group\": \"left_arm\",\"frame\":
\"waist_pitch_link\",\"use_cartesian_path\": false,\"left_motion_id\": 0,
\"left_waypoints\": [[0.26954124421585335, 0.4236418954013253,
0.10457946313346808, -0.1529562230220876, -0.6448902522919694,
-0.1707295374594559, 0.7290901051149286]]}'
```

```
data: []"

ros2 service call /dual_arm_waypoints cm_msgs/srv/CommonStreamRos2 "msg_id: ''
json_data: '{\"group\": \"right_arm\",\"frame\":\"waist_pitch_link\",
\"use_cartesian_path\": false, \"right_motion_id\": 0,\"right_waypoints\":
[[0.26954124421585335, -0.3736418954013253, 0.10457946313346808,
0.1529562230220876, -0.6448902522919694, 0.1707295374594559,
0.7290901051149286]]}'
data: []"


```

代码块

```
ros2 service call /dual_arm_waypoints cm_msgs/srv/CommonStreamRos2 "msg_id: ''
json_data: '{\"group\": \"dual_arm\",\"frame\":
\"waist_yaw_link\",\"left_motion_id\": 0, \"right_motion_id\": 0,
\"left_waypoints\": [[0.26954124421585335, 0.3736418954013253,
0.10457946313346808, -0.1529562230220876, -0.6448902522919694,
-0.1707295374594559, 0.7290901051149286],[0.28954124421585335,
0.3736418954013253, 0.10457946313346808, -0.1529562230220876,
-0.6448902522919694, -0.1707295374594559, 0.7290901051149286]],
\"right_waypoints\": [[0.26954124421585335, -0.3736418954013253,
0.10457946313346808, 0.1529562230220876, -0.6448902522919694,
0.1707295374594559, 0.7290901051149286],[0.28954124421585335,
-0.3736418954013253, 0.10457946313346808, 0.1529562230220876,
-0.6448902522919694, 0.1707295374594559, 0.7290901051149286]]}'
data: []"


ros2 service call /dual_arm_waypoints cm_msgs/srv/CommonStreamRos2 "msg_id: ''
json_data: '{\"group\": \"left_arm\",\"frame\":
\"waist_yaw_link\",\"use_cartesian_path\": false,\"left_motion_id\": 0,
\"left_waypoints\": [[0.26954124421585335, 0.4236418954013253,
0.10457946313346808, -0.1529562230220876, -0.6448902522919694,
-0.1707295374594559, 0.7290901051149286]]}'
data: []"

ros2 service call /dual_arm_waypoints cm_msgs/srv/CommonStreamRos2 "msg_id: ''
json_data: '{\"group\": \"right_arm\",\"frame\":\"waist_yaw_link\",
\"use_cartesian_path\": false, \"right_motion_id\": 0,\"right_waypoints\":
[[0.26954124421585335, -0.3736418954013253, 0.10457946313346808,
0.1529562230220876, -0.6448902522919694, 0.1707295374594559,
0.7290901051149286]]}'
data: []"

```

```
15
```

4.2.4.4 注意事项

1. 使用use_cartesian_path置为 true时，初始位置需要较为良好，且成功率较低；

4.2.5 执行上报服务名称

代码块

```
/waypoints_action_report
```

4.2.6 数据格式

代码块

```
ros2: cm_msgs::srv::CommonStreamRos2
```

4.2.6.1 请求数据格式

moveit异常上报error code : Moveit Error Code

代码块

```
request:
    msg_id: ''
    json_data:{
        "action_name": "motion_result",
        "left_motion_id": 0,
        "right_motion_id": 0,
        "result": 1    //成功=1 失败=0
        "error_code": 1  //详见error code 说明
    }
    data: []
```

4.2.6.2 响应数据格式

代码块

```
response:
    result: true //指令接收成功
```

4.3 障碍物

4.3.1 自定义添加环境障碍物

4.3.1.1 接口说明

当前接口基于 ROS Service 向运动控制接口上传/更新用户自定义的环境障碍物信息，包括立方体、球体等...

4.3.1.2 接口类型

ROS 2 Service

4.3.1.3 服务名称

代码块

```
/dual_arm_collision_object
```

4.3.1.4 数据格式

代码块

```
ros2: cm_msgs::srv::CommonStreamRos2
```

4.3.1.4.1 请求数据格式

代码块

```
request:
        msg_id: ''
        json_data:{
            "cmd": "add", //add or remove
            "sharp_type":"BOX", //BOX SPHERE
            "object_name": "box1",
            "dimensions": [], //dimensions of the shape
            "pose": [x, y, z, orientation_x, orientation_y, orientation_z,
orientation_w]
        }
        data: []
```

4.3.1.4.2 响应数据格式

代码块

```
response:
        result: true // true成功 false失败
```

4.3.1.4.3 使用案例

代码块

```
//增加box1
ros2 service call /dual_arm_collision_object cm_msgs/srv/CommonStreamRos2 "msg_id: ''
json_data: '{\"cmd\": \"add\",\"sharp_type\": \"BOX\",\"object_name\": \"box1\", \"dimensions\": [0.15, 0.34, 0.3], \"pose\": [0.7,-0.05,0.5,0,0,0,1]}'
data: []"

//增加sphere1
ros2 service call /dual_arm_collision_object cm_msgs/srv/CommonStreamRos2 "msg_id: ''
json_data: '{\"cmd\": \"add\",\"sharp_type\": \"SPHERE\",\"object_name\": \"sphere1\", \"dimensions\": [0.15], \"pose\": [0.7,-0.05,1.0,0,0,0,1]}'
data: []"

//删除sphere1
ros2 service call /dual_arm_collision_object cm_msgs/srv/CommonStreamRos2 "msg_id: ''
json_data: '{\"cmd\": \"remove\",\"object_name\": \"sphere1\"}'
data: []"

// 查看添加成功与否
方式1： 打开rviz，add MotionPlanning 插件
方式2： ros2 topic echo monitored_planning_scene
```

4.3.2 环境障碍物感知

4.3.2.1 接口说明

通过头部RGBD相机获取机器人环境点云信息，将点云转化为障碍物地图（Octomap）

4.3.2.2 接口类型

ROS 服务接口

4.3.2.3 服务名称

`/obstacle_map`

4.3.2.4 数据格式

`std_srvs::srv::Trigger`

4.3.2.4.1 请求数据格式

代码块

```
std_srvs::srv::SetBool
```

4.3.2.4.2 响应数据格式

代码块

```
std_srvs::srv::SetBool
```

4.3.2.4.3 使用案例

1. 启动本体

代码块

```
 ros2 launch body_control body.launch.py
```

2. 在orin上启动相机

代码块

```
需发布出原始点云话题 /camera/depth/points 以及对应TF数据作为外参
~~ros2 launch orbbec_camera gemini_330_series.launch.py enable_point_cloud:=true  publish_tf:=true~~
```

3. 障碍物感知

代码块

```
ros2 launch tianyi2_bringup tianyi2.launch.py gui:=true
ros2 launch tianyi2_bringup tianyi2_moveit.launch.py
ros2 control switch_controllers --deactivate empty_L_controller empty_R_controller
ros2 control switch_controllers --activate  moveit_left_arm_controller moveit_right_arm_controller

# 触发感知
ros2 service call /obstacle_map std_srvs/srv/SetBool "{data: true}"
# 取消感知
```

```
ros2 service call /clear_octomap std_srvs/srv/Empty
ros2 service call /obstacle_map std_srvs/srv/SetBool "{data: false}"
```

5. 真机操作接口

下层控制器主要用于直接操作硬件，在使用之前需要对各个子模块进行使能操作，所有对硬件的服务均运行于/EAIHardware这个节点上，其中包括使能、模式切换等。

备注：/EAIHardware这个节点仅存在于真机模式下，在启动时需要添加hardware:=real 这个参数，才能够启动真机模式

使能

例如：使能手臂

代码块

```
ros2 service call /EAIHardware/set_arm_enable std_srvs/srv/SetBool "{data: true}"
```

当使能手臂后，该框架会开始发布双臂的相应话题，若希望释放双臂资源给其他程序使用，则将其去使能

代码块

```
ros2 service call /EAIHardware/set_arm_enable std_srvs/srv/SetBool "{data: false}"
```

类似的，对腿、腰、头的使能为

代码块

```
ros2 service call /EAIHardware/set_leg_enable std_srvs/srv/SetBool "{data: true}"
ros2 service call /EAIHardware/set_waist_enable std_srvs/srv/SetBool "{data: true}"
ros2 service call /EAIHardware/set_head_enable std_srvs/srv/SetBool "{data: true}"
```

全部使能为

```
代码块
1   ros2 service call /EAIHardware/set_all_enable std_srvs/srv/SetBool "{data:
    true}"
```

注意：使能需要在控制器inactive的情况下进行，否则可能发生突然的剧烈运动

模式切换

下层控制算法主要为三个模式，高刚度模式、柔顺模式、拖动模式。

模式的切换需要通过更改mode实现，

```
代码块
1   ros2 service call /EAIHardware/set_arm_mode eai_manipulator_msgs/srv/Mode "
    {mode: 0}"
```

- mode:0 柔顺模式 （力位混合）
- mode:1 高刚度模式（速度环）
  - (版本V0.1) 当前测试发现因真机控制频率波动，导致速度环下发不稳定，会频繁中断与body_control的通信，因此优先考虑使用mode 3
- mode:2 重力补偿模式 （力位混合，kp kd 给0）
- mode:3 高刚度模式(位置环)

注意：有其他部分mode数值为合法值，但为内部保留，暂不对用户开放，请不要设置为0、1、2、3以外的其他数值

使用原则

不同上层控制器之间的切换通过ros2 control 框架来进行动态的插件切换，占用同一硬件资源的不同上层控制器不能同时工作，例如左手的关节空间控制器jointspace_arm_L_controller 与 左手的笛卡尔空间局部qp控制器endpose_single_arm_qp_L_controller不能够同时激活。

在切换上层控制器时需要先关闭以激活的控制器，否则将无法激活下一个控制器。

下层功能主要通过/EAIHardware/set_arm_mode这个服务进行切换；不同模式下有不同的控制参数可以调节；

6. 警告、错误信息

所有警告、错误信息都以如下接口进行发布

代码块

```
std_msgs/Header header

string level                    # 信息级别
uint32 code                     # 信息代码
string source                   # 发布源头
string info                     # 信息内容
```

发布的话题为 /manipulator/warn_and_error_info

消息类型 eai_manipulator_msgs/msg/WarnAndErrorInfo

source 为消息源，根据此字段可定位警告与错误源

code 为信息号

info 为基本提示信息

level 为信息级别，包含主要为

- warn 警告信息，一般为提示作用，该类型说明当前出现的问题为一些不太符合预期的操作，但仍然处于正常可用
  - 例如：运动达到最大限速、目标位姿超出判断阈值等
- error 错误信息， 一般为报错，该类型信息出现时，系统不能正常工作
  - 例如：发送的关节运动角度超过最大限位，为非法值

应用层在使用中需要监控/manipulator/warn_and_error_info这个话题，以获得警告和报错的相关信息。

应做到在运行中尽可能降低warn信息的发生次数，以及不发生error信息。

7. 参数设置

本框架中的所有上层控制器、下层控制器，参数均采用标准ros2节点参数，可通过ros2 param list获取参数列表，上层控制器的参数，其位于控制器的节点名之下；

8. 操作范例

整体启动：

```
代码块
1   ros2 launch tianyi2_bringup tianyi2.launch.py gui:=True hardware:=sim
```

gui:=True 会打开rviz

hardware:=sim 启动的是仿真模式，不会与硬件产生通信

hardware:=real 启动的是真机模式，只有在真机模式下才会出现/EAIHardware节点

启动后会默认激活各个部位的empty控制器，使得机器人保持在原位

注意：

如果使用远程登录例如mobaxterm，通过远程发送方式将rviz的可视化界面传输到本地，会占用较高cpu负载导致系统中其他部分性能收到影响。已知的影响包括但不限于：

- 摄像头帧率下降
- 控制频率下降

控制器切换

1. 停止正在运行的控制器

```
代码块
1   ros2 control switch_controllers --deactivate empty_L_controller    # 关闭左手
    empty控制器
```

2. 激活需要运行的控制器

```
代码块
1   ros2 control switch_controllers --activate jointspace_arm_L_controller # 激活左
    手关节空间控制器
```

3. 下发相应topic或service

代码块

```
# 将左手运动到各个关节的中间位置
ros2 topic pub /jointspace_commands_L std_msgs/msg/Float64MultiArray 'data:
[0.0, 1.18, 0.0, -1.3, 0.0, -0.13, 0.18]' --once
```

4. 关闭该控制器，启动其他控制器进行后续操作

8.1 详细举例

左手局部末端控制

末端控制需要下发话题**/endposetarget_L**，本例使用tools/script/left_target.py使用交互式marker下发

代码块

```
# 先使用关节控制器运动到中间位置
ros2 control switch_controllers --activate jointspace_arm_L_controller
ros2 topic pub /jointspace_commands_L std_msgs/msg/Float64MultiArray 'data:
[0.0, 1.18, 0.0, -1.3, 0.0, -0.13, 0.18]' --once
# 将marker拖动到距离左手末端较近位置和姿态
ros2 control switch_controllers --deactivate jointspace_arm_L_controller --
activate endpose_single_arm_qp_L_controller
```

9. 自动控制器切换

代码块

```
/jointspace_commands_L
/jointspace_commands_R
/jointspace_commands_L_collisionfree
/jointspace_commands_R_collisionfree
/jointspace_commands_Dual
/jointspace_commands_body
/jointspace_commands_head

/endposetarget_L
/endposetarget_R
```

```
/endposetarget_Dual
/endposetarget_Dual_wholebody
/endposetarget_Body
/endposetarget_Head
```

自动切换：

接口与控制器对应关系

话题名	控制器名
/jointspace_commands_L	jointspace_arm_L_controller
/jointspace_commands_R	jointspace_arm_R_controller
/jointspace_commands_L_collisionfree	jointspace_single_arm_qpik_L_controller
/jointspace_commands_L_collisionfree	jointspace_single_arm_qpik_R_controller
/jointspace_commands_Dual	jointspace_ruckig_qp_controller
/jointspace_commands_body	jointspace_body_controller
/jointspace_commands_head	jointspace_head_controller
/endposetarget_L	endpose_single_arm_qp_L_controller
/endposetarget_L	endpose_single_arm_qp_R_controller
/endposetarget_Dual	endpose_dual_arm_qp_controller
/endposetarget_Dual_wholebody	endpose_wholebody_controller
/endposetarget_Body	endpose_body_controller
/endposetarget_Head	endpose_head_controller

ros2 contorl 命令行工具:

代码块

```
ros2 control auto_switch_mode --enable #开启自动切换模式
ros2 control auto_switch_mode --disable #关闭自动切换模式
```

对应服务名为:

```
代码块
1   /controller_manager/auto_switch_mode
```

类型为:

```
代码块
1   std_srvs/srv/SetBool
```

带冲突解决的手动切换

ros2 control activate_controllers 后面跟需要激活的控制器

```
代码块
1   ros2 control activate_controllers endpose_single_arm_qp_L_controller
    endpose_single_arm_qp_R_controller
```

当执行手动切换后，自动切换模式会关闭。想要打开，需要使用ros2 control auto_switch_mode --enable

对应服务为

```
代码块
1   /controller_manager/activate_controller
```

服务类型

```
代码块
1   controller_manager_msgs/srv/SwitchController
```

该服务接口中只有 activate_controllers 字段有效，其他字段为接口兼容性考虑

使用说明

自动切换遵循以下几条规则

1. 根据收到的topic自动切换为对应的控制器。
2. 相互冲突的控制器，激活最后收到topic的那个
3. 切换检测的周期为0.1秒，上一个topic停止发送后，需要延迟至少0.1秒(建议值0.2秒)以上，否则将导致切换冲突此次切换失败
4. 自动切换的时机为topic收到的上升沿
5. topic停止发送后，控制器不会自动关闭
6. 使用手动切换ros2 control activate_controllers后自动切换会关闭

10. 信息、警告、报错编码表

info级别信息

编码	信息源source	解释	处理
100101		内部规划器状态发生变化时发生，一般用来指示收到新的目标数据	
100102		内部规划器运行完成时发生，可用于指令的到位判断	

warn级别警告

编码	信息源source	解释	处理
300100		给定目标位姿超出跟踪限	
30025x		关节达到最小限位	
30026x		关节达到最大限位	

error级别报错

编码	信息源source	解释	处理
600100	error	内部规划器运行报错，通常为优化问题不可行	
600101	error	关节指令超上限位	
600102	error	关节指令超下限位	
600120	error	from_frame设置错误	
600130	error	to_frame设置错误	